

Introducción a la Programación

Clase 00: Introducción a Python

Daniela Opitz

daniela.opitz@usm.cl



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Clase de Hoy

- Presentaciones
- Horarios
- Reglas de la asignatura
- Introducción a Python
- Primer Programa 'Holamundo'
- Variables y preámbulo a operaciones

Presentaciones

Daniela Opitz

Académica

Departamento de Informática

Universidad Técnica Federico Santa María

PhD. Física UNSW Sydney

Lic. Astronomía UChile

contacto

daniela.opitz@usm.cl

Oficina: K-605

Departamento de Informática

Presentaciones



Introducción a la Programación

- Curso de nivel básico de programación
- Lenguaje de programación: Python



Horarios

- **Cátedra:**

- Lunes: 12:30 - 13:40 hrs
- Miércoles: 12:30 - 13:40 hrs

- **Hora de consultas:**

- Lunes: 14:40 - 15:50 hrs

- **Otras consultas**

- Escribir a daniela.opitz@usm.cl

Reglas (Evaluaciones: 5 tipos)

Evaluación	¿Dónde se rinde?	¿Cómo?	% nota
Certámenes [C]	Sala de certamen (horario especial: 17:30 hrs)	En papel, 2 horas, presencial	50%
Controles [CTL]	Laboratorio (en su paralelo)	En AULA, al inicio de la sesión, ~10 min	20%
Tareas [T]	Ayudantía	Entregas parciales + quiz o interrogación	20%
Ev. formativas [EF]	Online (AULA)	Asincrónicas, antes de cada unidad, intentos ilimitados	5%
Ej. autocorregidos [ECA]	Online (AULA)	Juez automático, durante cada unidad	5%

Todas las evaluaciones son estrictamente individuales.

Reglas (Estructura de la Asignatura)

El curso está estructurado en **8 unidades de aprendizaje**:

Certamen 1

1. Programas secuenciales

2. Decisiones

3. Ciclos

4. Strings

5. Funciones

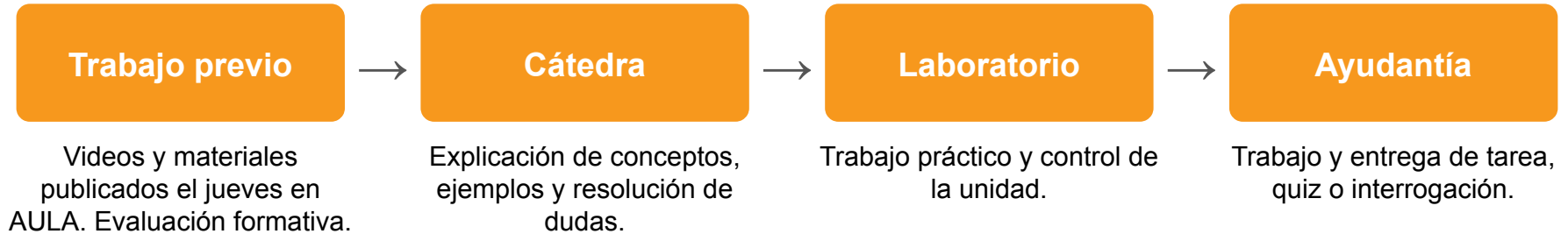
Certamen 2

6. Listas

7. Diccionarios

8. Texto y archivos

Reglas (Desarrollo de una Unidad)



Durante el semestre: 2 certámenes presenciales en papel

Reglas (Ponderaciones)

C = Certámenes en papel (50%)

CTL = Controles en laboratorio (20%)

T = Tareas en ayudantía (20%)

EF = Evaluaciones formativas (5%)

ECA = Ejercicios de corrección automática (5%)

Promedio ponderado

$$P = C \times 0.50 + CTL \times 0.20 + T \times 0.20 + EF \times 0.05 + ECA \times 0.05$$

Cada componente es el promedio aritmético de las evaluaciones de ese tipo.

Reglas (Nota Final)

Para aprobar la asignatura, el promedio de certámenes (C) debe ser al menos 55.

Si $C \geq 55$:

Nota Final = P

Si no:

Nota Final = $\min(C, 54)$ (queda limitada por el promedio de certámenes)

Como en todas las asignaturas, para aprobar se debe tener Nota Final ≥ 55 .

Reglas (Certámenes)

El curso contempla dos certámenes y un certamen recuperativo. Todos se realizan en papel, comienzan a las 17:30 horas y tienen una duración de dos horas.

- **Certamen 1:** Jueves 8 de octubre
- **Certamen 2:** Jueves 19 de noviembre
- **Certamen Recuperativo:** Jueves 3 de diciembre

El certamen recuperativo lo puede rendir **cualquier estudiante**, pero se debe tener en cuenta que **siempre** reemplazará la menor nota entre el C1 y el C2, aunque esto implique rebajar la nota.

El certamen recuperativo puede reemplazar sólo una nota.

Reglas (Controles)

Se realiza **un control por cada unidad** del curso. Buscan verificar que se han adquirido los conocimientos básicos necesarios para abordar las actividades prácticas de la unidad.

- Se realizan generalmente en **AULA**, al inicio de la sesión de laboratorio.
- Tienen una duración aproximada de **10 minutos**.
- Deben rendirse **presencialmente**. La ausencia implica **nota 0** en el control de ese día.
- Sólo pueden rendirse en el **paralelo de laboratorio** en el que el o la estudiante está inscrito(a).

Reglas (Evaluaciones Formativas)

- Junto con los videos, se habilitan cuestionarios en AULA antes del inicio de cada unidad.
- Buscan verificar que se comprendieron los contenidos de los videos y demás materiales de estudio.
- Se responden de forma **asincrónica y remota**.
- Tienen intentos ilimitados dentro del plazo establecido.
- Para obtener el puntaje máximo, todas las respuestas deben ser correctas.
- Generalmente disponibles desde el **jueves** previo al inicio de la unidad hasta el **domingo a las 23:55 horas**.

Reglas (Ejercicios Autocorregidos)

- En cada unidad se habilitará en AULA un conjunto de ejercicios de programación.
- Las soluciones serán evaluadas mediante un **juez automático**, que verificará su corrección en base a casos de prueba, algunos de los cuales son secretos.
- Los ejercicios tendrán **distintos niveles de dificultad**.
- Para obtener el puntaje total, se debe resolver correctamente **al menos un ejercicio en cada unidad**.
- Se recomienda resolver **la mayor cantidad posible**: permite practicar más y enfrentar problemas de dificultad creciente.

Reglas (Tareas)

- Se desarrollan **cuatro tareas**, cada una asociada a una o más unidades del curso.
- Se realizan **exclusivamente** durante las sesiones de ayudantía.
- Excepción en la primera tarea, que tiene una primera parte asincrónica.
- La planificación de las tareas está disponible en la programación publicada en AULA.

Objetivo: desarrollar problemas de programación de mayor complejidad, integrando progresivamente los contenidos estudiados en las distintas unidades del curso.

Reglas (Tareas: Estructura)

Cada tarea puede estar compuesta por varias etapas, dependiendo de sus objetivos:

- **Entrega conceptual**, centrada en la comprensión del problema y el diseño preliminar de la solución.
- **Entregas parciales**, donde se implementan distintas partes de la solución. Cada entrega se construye sobre la anterior.
- **Quiz o interrogación oral** al finalizar la tarea, para verificar la comprensión de la solución desarrollada.

Asistencia: las actividades de la tarea deben realizarse en la sesión correspondiente. La ausencia implica **nota 0** en la parte que debía desarrollarse ese día.

Integridad del Proceso de Evaluación

Todas las evaluaciones de la asignatura son **estrictamente individuales**. Las soluciones deben ser de autoría absoluta del o de la estudiante.

- No trabajar en equipo para construir soluciones.
- No pedir a terceros que escriban código.
- No obtener código de fuentes externas.

Responsabilidad compartida: también constituye falta facilitar soluciones a otros estudiantes. Es responsable quien comete la falta y quien ayuda a cometerla.

Uso de IA Generativa

Dada la naturaleza y objetivos de esta asignatura, está **estrictamente prohibido** utilizar herramientas de Inteligencia Artificial Generativa para construir soluciones de tareas, controles o certámenes.

El objetivo del curso es que cada estudiante desarrolle sus propias habilidades de programación: usar IA para producir soluciones reemplaza el proceso de aprendizaje y representa un camino más corto para hacer trampa, entregando soluciones que no son de la autoría del o de la estudiante.

Las soluciones generadas o asistidas por IA se considerarán una **falta grave (fraude)**.

Sanciones Específicas

Tareas

- Nota 0 en la tarea completa.
- Informe a la jefatura de carrera correspondiente.
- Registro según los procedimientos institucionales.

Certámenes

- Se deriva el caso a la Comisión Universitaria correspondiente, según lo que establece el Reglamento de Deberes, Derechos y Disciplina del Estudiantado.

Cómo Pedir Ayuda Correctamente

A una IA o a otra persona

Sí corresponde

- Preguntar por conceptos.
- Mostrar errores y explicar qué intentaste.
- Pedir orientación sobre cómo depurar.
- Discutir ideas generales.

No corresponde

- Pedir o entregar soluciones completas.
- Copiar código de otra persona.
- Reutilizar código externo sin autorización.
- Usar IA para generar la solución evaluada.

¿Con Quién Puedo Contar?

Equipo docente

- **Profesor(a) de cátedra:** responsable del desarrollo de la asignatura.
- **Profesor(a) de laboratorio:** guía el trabajo práctico y atiende consultas.
- **Ayudante:** apoya la resolución de las tareas y responde preguntas.
- **Coordinación de la asignatura:** situaciones excepcionales y lo que el equipo docente no consiga resolver.

Canales de apoyo

- **Horarios de atención:** puedes asistir al horario de atención de cualquier profesor(a) del curso, de forma presencial o virtual.
- **Foro de consultas en AULA:** espacio oficial para preguntas sobre los contenidos y las actividades del curso. Antes de publicar, revisa si ya fue respondida.

No esperes hasta el último momento para pedir ayuda. Si tienes dudas, ¡consúltanos oportunamente!

Resumen de Reglas Clave

- Las evaluaciones combinan certámenes, controles, tareas, cuestionarios de evaluación formativa y ejercicios autocorregidos.
- Para aprobar, el promedio de certámenes debe ser al menos 55.
- Las tareas combinan diversos elementos y se construyen incrementalmente en las ayudantías.
- Toda evaluación es individual.
- El uso de IA generativa para producir soluciones evaluadas está prohibido.

Contenidos

Ocho unidades:

- 1. Programas secuenciales**
- 2. Flujo condicional (decisiones)**
- 3. Flujo iterativo (ciclos)**
- 4. String**
- 5. Funciones**

CERTAMEN 1

- 6. Listas**
- 7. Diccionarios**
- 8. Procesamiento de texto y archivos**

CERTAMEN 2

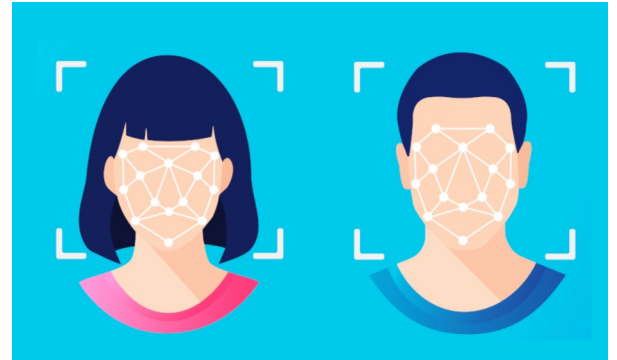
¿Para qué sirve la programación?

Algoritmo

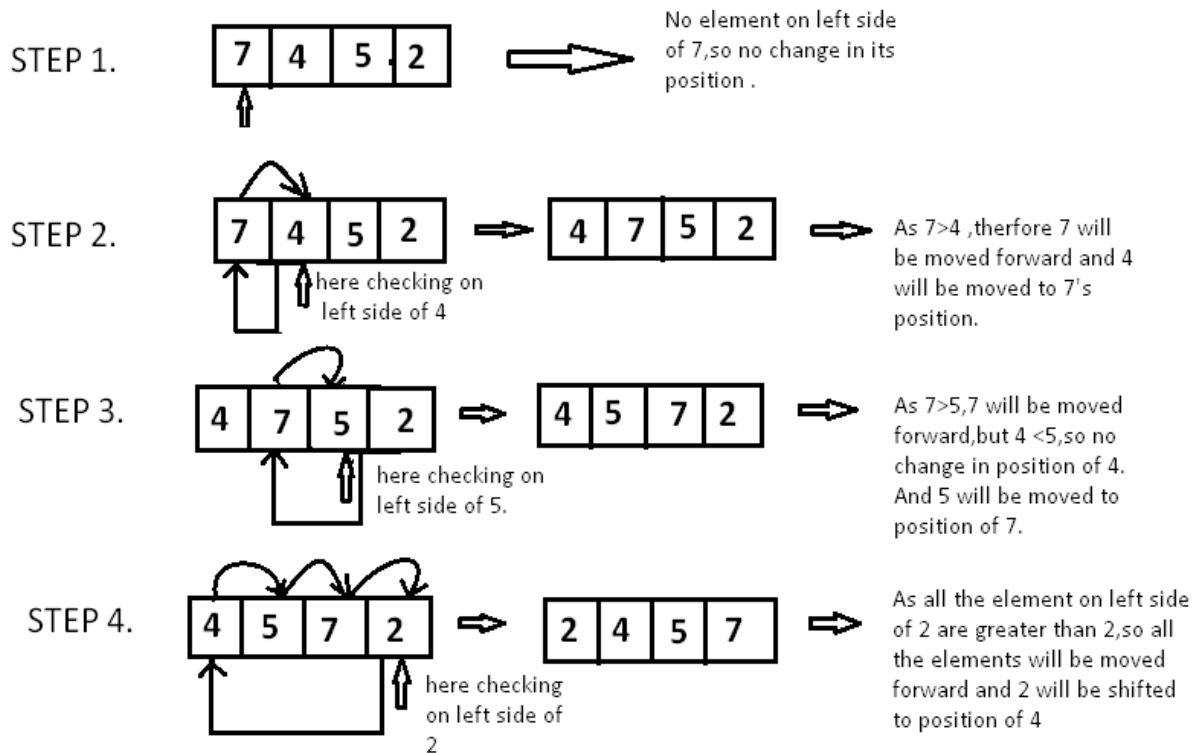
Conjunto de instrucciones (finitas) que deben seguirse por orden para ejecutar una tarea.

Ej:

- receta de cocina
- algoritmo para girar dinero en cajero automático
- algoritmo de reconocimiento facial



Algoritmo



Lenguajes de Programación

Lenguaje formal creado para darle instrucciones a un computador.



Python



<https://www.python.org/downloads/>

Hola mundo

```
# holamundo.py  
print('¡Hola mundo!')
```



```
2. bash  
$ python3 holamundo.py  
Hola mundo!  
$
```

Variables

¿Qué es una variable?

Memoria que permite almacenar datos

Ejemplo

```
edad = 37  
print('Edad', edad)  
37
```

Memoria

¿Qué entendemos por memoria?

Dispositivo electrónico que permite almacenar datos durante un intervalo de tiempo



1	3	4	4	6	7	0	1
---	---	---	---	---	---	---	---

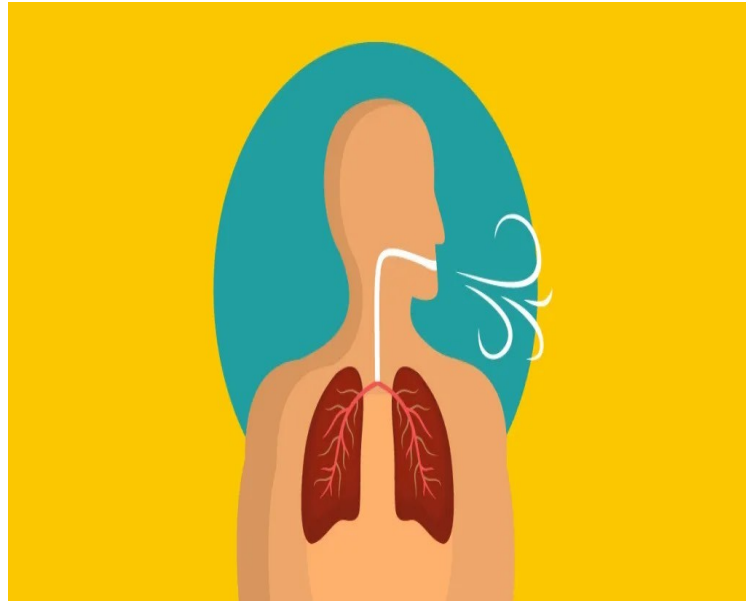
Tipos de Datos

- `int`: representa números **enteros** (ej. 3)
- `float`: representa números **reales** (ej. 3.27)
- `bool`: representa valores **boolean** (**True** y **False**)
- `str`: representa valores **strings** (texto)
- `NoneType`: representa el valor **None**

Puedes usar `type()` para ver el tipo

Problema

- Una persona respira entre 12 y 20 veces por minuto. ¿Si actualmente son las 12 del día, cuántas veces he respirado hoy?



Identificadores

- Nombre que le puedo dar a una variable o a una función
- Deben comenzar con una letra o con un guión bajo
- Pueden contener dígitos y guión bajo
 - x, nombre_apellido, y55, _holahola
- Pero no pueden comenzar con números o contener caracteres de operación:
 - 12, 1x, -55, 33mineros, micro\$oft
- Tampoco pueden ser palabras reservadas:
 - Ejemplo: `int`, `if`, `return`, `float`...

Identificadores (Palabras Reservadas)

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Identificadores

- ¡Deben ser informativos!
 - Siglas son confusas: `LAX`, `CCP`, `myvar`
 - Evitar uso de nombres largos: `mi_variable_de_tipo_string`
 - Recuerde que su código será leído por otra persona... incluido su "futuro yo"
- Buenos ejemplos:

`edad`, `suma_parcial`, `temperatura`, `process_type`, `current_time`